

NOM: ALIAWY MFAM SAKER RICHLAND

COMPTE RENDU – TP ANALYSE DU TRAFIC RESEAU AVEC WIRESHARK

1. INTRODUCTION

Wireshark est un analyseur de paquets réseau. Il permet de capturer et d'inspecter en temps réel les échanges entre machines. En cybersécurité, l'analyse réseau est essentielle pour détecter des comportements anormaux (scans, attaques, exfiltration). Ce TP vise à comprendre ce qu'un attaquant génère réellement sur le réseau lorsqu'il utilise Nmap, et comment un analyste SOC peut l'identifier.

2. DESCRIPTION DES MANIPULATIONS

Matériel utilisé :

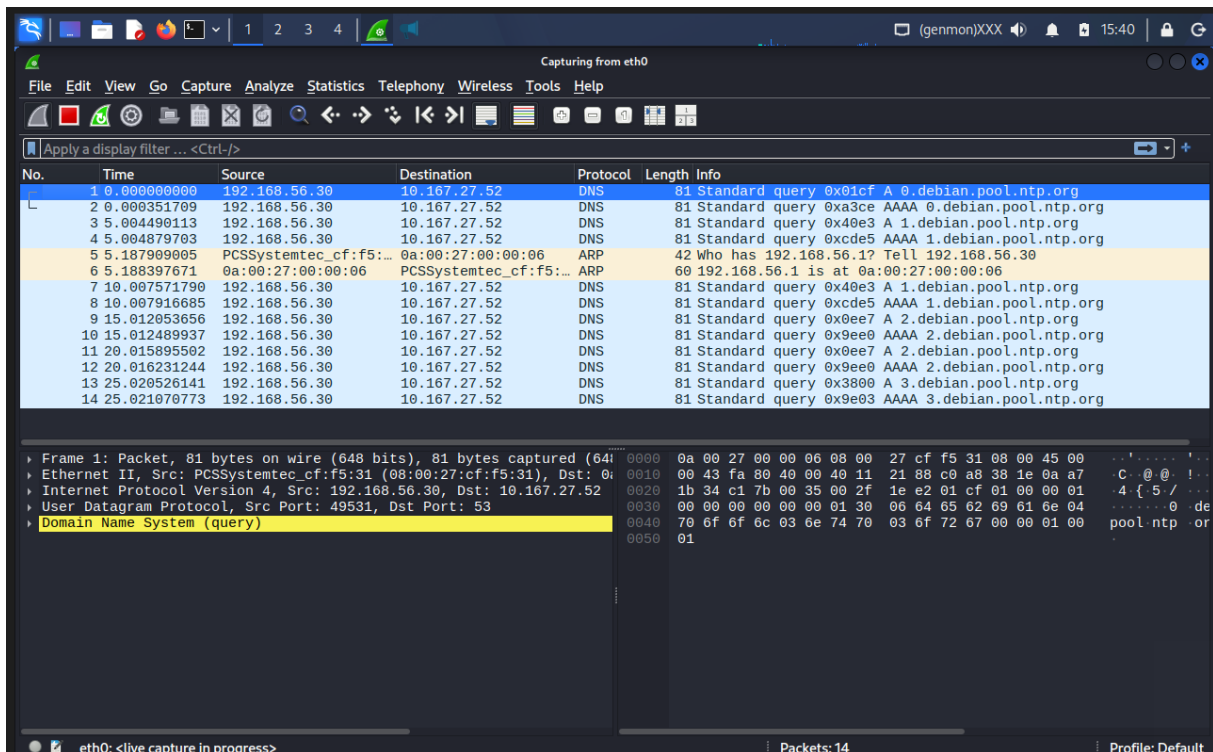
- Kali Linux (attaquant) – IP : 192.168.56.30
- Ubuntu (cible) – IP : 192.168.56.10
- Wireshark et Nmap sur Kali

Étapes réalisées :

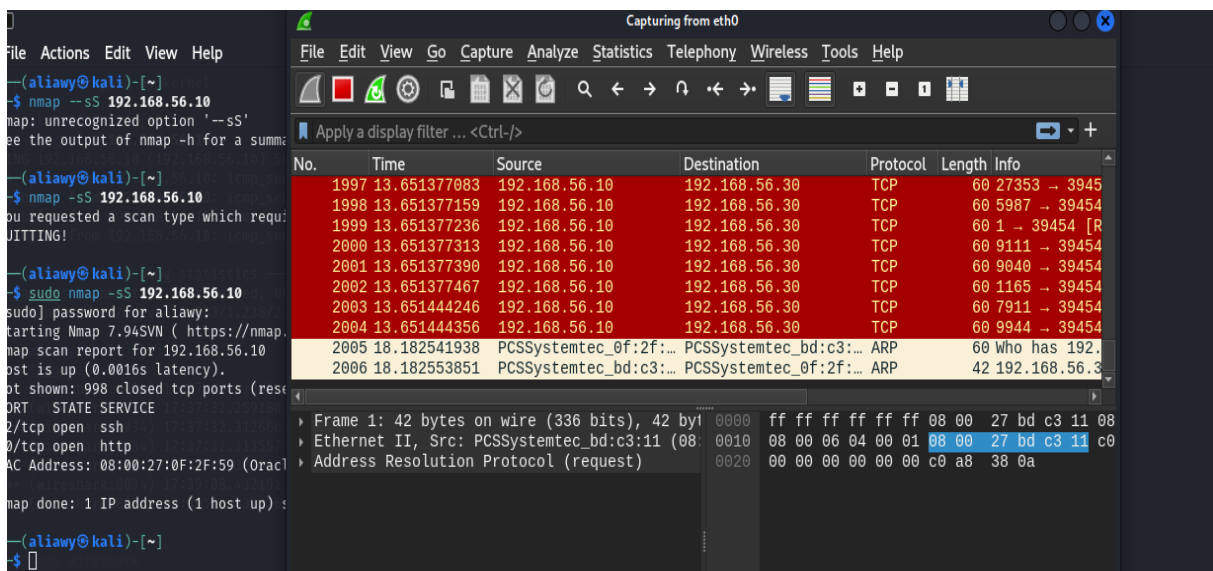
1. Lancement de Wireshark avec `sudo wireshark` pour avoir les droits de capture.
2. Sélection de l'interface `eth0` (celle du réseau Host-Only, IP 192.168.56.30).
3. Démarrage de la capture.
4. Génération d'un scan SYN avec la commande `nmap -sS 192.168.56.10` dans un second terminal.
5. Arrêt de la capture après le scan.
6. Filtrage des paquets SYN avec `tcp.flags.syn == 1`.

7. Analyse détaillée de quelques paquets pour observer les flags TCP (SYN, SYN-ACK, RST).

3. RESULTATS OBTENUS (AVEC CAPTURES D'ECRAN)

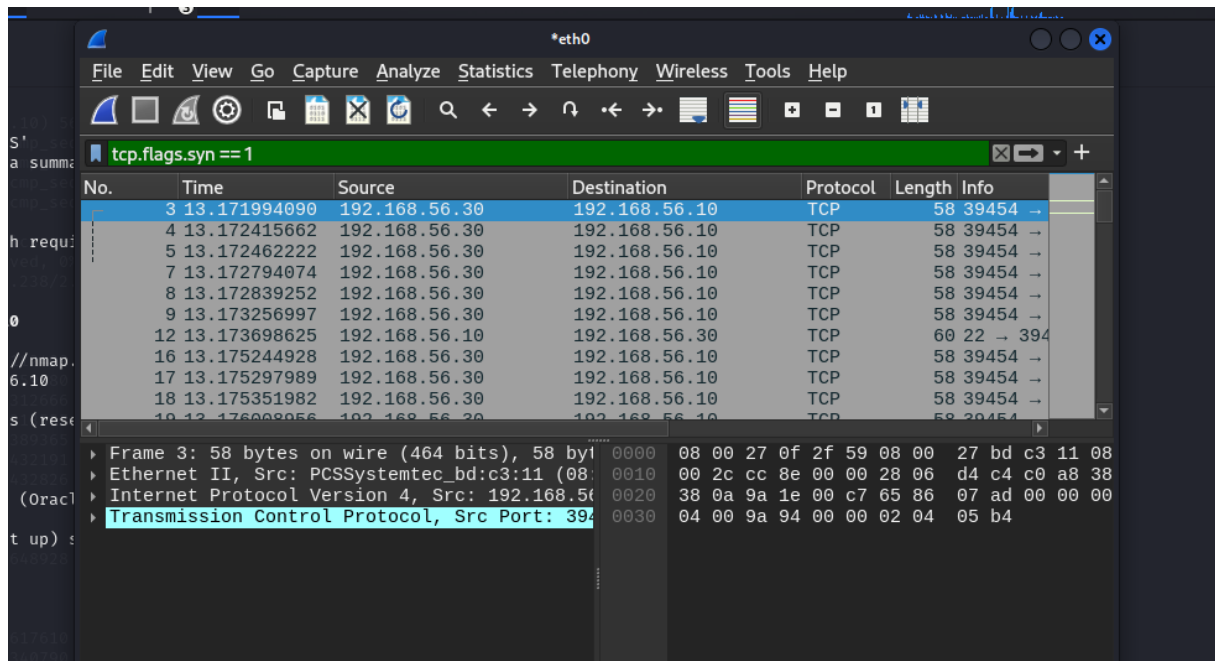


Partie 1 : Lancement de wireshark et capture du trafic



Partie2 génération d'un trafic scan

On observe une rafale de paquets entre Kali (192.168.56.30) et Ubuntu (192.168.56.10). La majorité sont des segments TCP avec flag SYN.

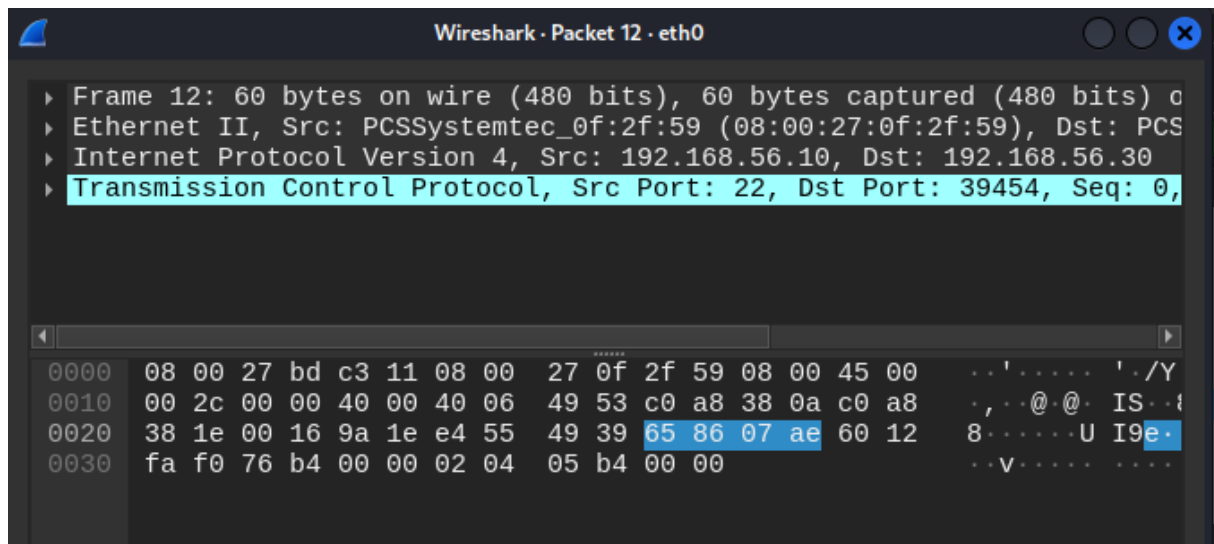


No.	Time	Source	Destination	Protocol	Length	Info
3	13.171994090	192.168.56.30	192.168.56.10	TCP	58	39454 →
4	13.172415662	192.168.56.30	192.168.56.10	TCP	58	39454 →
5	13.172462222	192.168.56.30	192.168.56.10	TCP	58	39454 →
7	13.172794074	192.168.56.30	192.168.56.10	TCP	58	39454 →
8	13.172839252	192.168.56.30	192.168.56.10	TCP	58	39454 →
9	13.173256997	192.168.56.30	192.168.56.10	TCP	58	39454 →
12	13.173698625	192.168.56.10	192.168.56.30	TCP	60	22 → 39454
16	13.175244928	192.168.56.30	192.168.56.10	TCP	58	39454 →
17	13.175297989	192.168.56.30	192.168.56.10	TCP	58	39454 →
18	13.175351982	192.168.56.30	192.168.56.10	TCP	58	39454 →
19	13.176000056	192.168.56.30	192.168.56.10	TCP	58	39454 →

Frame 3: 58 bytes on wire (464 bits), 58 bytes captured (464 bits) on interface eth0	0000	08 00 27 0f 2f 59 08 00 27 bd c3 11 08
Ethernet II, Src: PCSSystemtec_bd:c3:11:08:00:27:bd:c3, Dst: 08:00:27:0f:2f:59:08:00	0010	00 2c cc 8e 00 00 28 06 d4 c4 c0 a8 38
Internet Protocol Version 4, Src: 192.168.56.30, Dst: 192.168.56.10	0020	38 0a 9a 1e 00 c7 65 86 07 ad 00 00 00
Transmission Control Protocol, Src Port: 39454, Dst Port: 22	0030	04 00 9a 94 00 00 02 04 05 b4

Partie3: filtrage des paquets

Seuls les paquets contenant le flag SYN sont affichés. Cela correspond exactement aux tentatives de connexion initiées par Nmap.



Partie4 analyse du fonctionnement tcp

Détail d'un échange TCP (port ouvert et port fermé)

. Interprétation des captures

- la partie 2 montre un comportement anormal : un grand nombre de paquets SYN vers des ports différents en très peu de temps. Un trafic légitime ne présente pas cette répétition rapide.
- la partie 3 isole efficacement les paquets suspects (les SYN). Cela permet à un analyste de ne pas être noyé dans le trafic.
- la partie 4 illustre le fonctionnement du scan SYN : Nmap n'établit jamais la connexion complète (il répond par RST après un SYN-ACK). Cela rend le scan plus furtif mais reste détectable par la répétition des SYN.

4. ANALYSE TECHNIQUE

Comment détecter un port ouvert ?

Un port est ouvert lorsque la cible répond à un SYN par un SYN-ACK. Dans Wireshark, on voit la paire SYN → SYN-ACK.

Comment reconnaître un scan réseau ?

- Une succession rapide de paquets SYN (ou d'autres flags) vers une même destination.
- Les ports sont sondés de manière séquentielle ou aléatoire.
- Absence de flux applicatif complet (pas de ACK final, pas de données).
- Le filtre `tcp.flags.syn == 1` permet de visualiser l'ensemble des tentatives.

6. UTILISATION AVANCEE DE WIRESHARK (RECHERCHE PERSONNELLE)

Voici trois filtres avancés utiles en analyse :

Filtre Utilité Exemple concret

`ip.addr == 192.168.56.10` Isoler tout le trafic vers/depuis une IP donnée Voir uniquement ce que la cible reçoit ou émet

`tcp.port == 80` Afficher le trafic HTTP Analyser des requêtes web vers un serveur

`tcp.flags.reset == 1` Détecter les paquets RST Repérer des ports fermés ou des connexions interrompues brutalement

6. LIEN AVEC LA CYBERSECURITE EN ENTREPRISE

Dans un SOC (Security Operations Center), les analystes utilisent Wireshark pour :

- Analyser en profondeur une alerte remontée par un IDS (ex : Snort, Suricata).
- Extraire des preuves lors d'une investigation post-attaque.
- Valider si un trafic est malveillant ou légitime.

Un scan réseau est détecté par la multiplicité des SYN en peu de temps, la diversité des ports et l'absence d'échange complet. Les outils de détection (Zeek, IDS) peuvent générer une alerte "Port Scan" lorsqu'un seuil est dépassé.

7. CONCLUSION

Ce que j'ai appris :

- Un scan Nmap n'est pas une "commande magique" mais une suite de paquets TCP bien précis.
- Les flags SYN, SYN-ACK, RST permettent de déterminer l'état des ports.
- Wireshark est un outil puissant pour visualiser concrètement une attaque réseau.

Difficultés rencontrées :

- Problème de permissions : Wireshark nécessite sudo ou l'appartenance au groupe wireshark.
- Confusion entre les interfaces réseau (eth0 vs eth1).
- Le scan SYN est si rapide qu'il faut bien synchroniser la capture et le scan.

Limites de Wireshark :

- Ne détecte pas automatiquement une attaque (pas d'alerte).
- Capture passive uniquement ; ne bloque pas le trafic.
- En environnement chiffré (TLS), le contenu des paquets n'est pas lisible.